

Bottleneck-Driven Optimization of Semantic Segmentation on Jetson Nano

Haotian Lei
Columbia University
hl3945@columbia.edu

Jiaming Liu
Columbia University
jl7250@columbia.edu

Abstract—Deploying semantic segmentation models on edge devices such as the NVIDIA Jetson Nano 2GB faces significant challenges due to limited compute capability, memory capacity, and memory bandwidth. Instead of blindly applying model compression methods, we adopt a bottleneck-driven approach: we first profile the complete inference pipeline to identify the actual runtime bottlenecks, and then apply targeted optimizations based on the profiling results.

Using DeepLabV3-MobileNetV3 as the baseline model (11.02M parameters, achieving 49.5% mIoU on the Cityscapes dataset), we conduct phase-level latency analysis and CUDA operation cost analysis on the Jetson Nano. The results show that the preprocessing stage accounts for 63% of the total runtime, while convolution operations dominate GPU execution time under severe memory bandwidth constraints.

We further explore two optimization directions: (1) resolution scaling, where the model achieves the best performance at its training resolution (256×512 , 8.0 FPS), while both lower and higher resolutions lead to accuracy degradation; and (2) FPGM structured pruning, which achieves up to $2.63 \times$ latency speedup with a 95.9% parameter reduction, at the cost of more noticeable accuracy loss.

Our analysis shows that on bandwidth-constrained edge hardware, the FLOPs reduction brought by pruning cannot be linearly translated into inference speedup. At the same time, resolutions higher than the training resolution provide no additional accuracy benefits.

Index Terms—semantic segmentation, edge inference, Jetson Nano, resolution scaling, structured pruning, bottleneck analysis

I. INTRODUCTION

A. Background

Semantic segmentation assigns category labels to each pixel in the image, so it becomes a basic task in applications such as autonomous driving, scene understanding and robot navigation [1]. Although the large model has achieved excellent accuracy performance on benchmark datasets, its deployment on resource-limited edge platforms still faces challenges. Devices such as NVIDIA Jetson Nano can provide GPU-accelerated inference and have the advantages of small size and low power consumption, but its Maxwell-architecture GPU, limited DRAM capacity and narrow memory bandwidth impose strict constraints on model throughput. Therefore, to achieve a balance between semantic segmentation accuracy and real-time edge performance, it is necessary to carefully analyze where inference time is consumed.

B. Problem Statement and Research Motivation

It is not easy to achieve real-time semantic segmentation on Jetson Nano 2GB. The device is equipped with a 128-core Maxwell GPU, and the shared CPU-GPU memory bandwidth is only 12.8 GB/s, which is far lower than the bandwidth level of datacenter GPUs. Therefore, the traditional model selection method based on parameter count or floating-point operations (FLOPs) is not completely reliable: a model with fewer FLOPs may not be able to obtain faster inference speed if it relies more on memory bandwidth than computing power.

The difference between the theoretical computation cost and the actual device performance prompts us to adopt the “profiling-first” method: before deciding on the optimization direction, first measure the actual runtime cost of the model on the real device.

C. Goals and Contributions

The main contributions of this article are as follows:

- 1) We systematically analyzed the end-to-end inference pipeline of DeepLabV3-MobileNetV3 on Jetson Nano, including performance analysis at the stage level (pre-processing, inference, post-processing) and CUDA operation level, so as to determine the main runtime bottleneck.
- 2) We conducted a systematic resolution scaling experiment on five input resolutions (128×256 to 768×1536) to analyze the latency-accuracy trade-off relationship under different resolutions.
- 3) We apply FPGM structured pruning under four pruning ratios (30%–90%), and prove that in a hardware environment with limited bandwidth, the speedup effect of pruning shows obvious sublinear characteristics.

Fig. 1 illustrates the overall methodology pipeline.

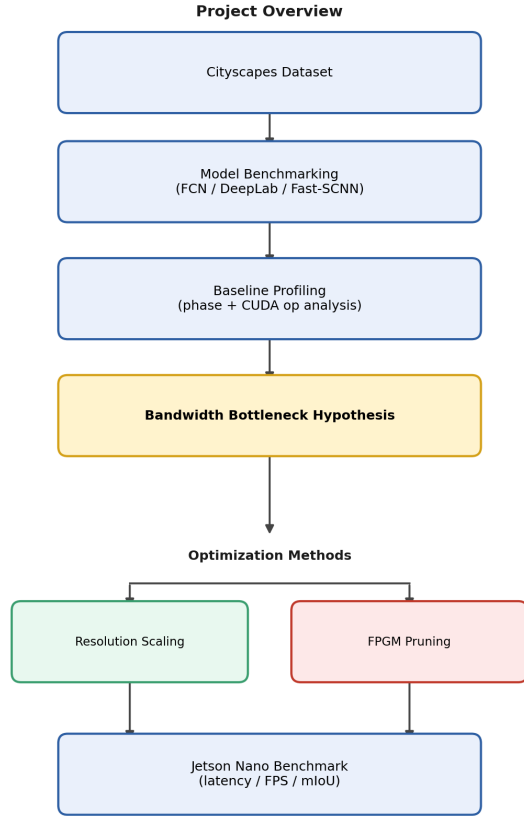


Fig. 1. Methodology overview. Starting from the Cityscapes dataset, we benchmark candidate models, profile the selected baseline to identify the bandwidth bottleneck, and then apply two targeted optimizations: resolution scaling and FPGM pruning.

II. RELATED WORK

A. Efficient Segmentation Architecture

The early semantic segmentation model based on Fully Convolutional Networks (FCN) [1] can achieve higher accuracy, but the large-scale ResNet backbone network it uses is too slow for real-time edge deployment. DeepLabV3 [2] improves segmentation accuracy through atrous convolution and Atrous Spatial Pyramid Pooling (ASPP), while combining lightweight backbone networks such as MobileNetV3 [3] to reduce the number of parameters. Fast-SCNN [4] achieves a very high throughput rate by replacing the complete encoder-decoder structure with a shared feature extractor and multi-branch classification head, but compared with DeepLabV3, it will lose some accuracy. MobileNetV2 [5] proposes an inverted residual with linear bottleneck, which becomes an important foundation of the backbone networks used in MobileNetV3 and DeepLabV3-MNV3.

B. Resolution Scaling

Input resolution is an important means to achieve a trade-off between accuracy and speed. EfficientDet [6] demonstrates a composite scaling method of resolution, depth and width, thus covering a wider range of efficiency frontiers. However, when the model performs inference at a different resolution from the training stage, the relationship between resolution and accuracy is non-monotonic: the accuracy of the model usually decreases when it is higher or lower than the training resolution. Therefore, practical testing and empirical evaluation are required for each target resolution.

C. Structured Pruning

Structured pruning generates a denser and hardware-friendly model by removing the entire convolutional filter without relying on sparse arithmetic support. Li et al. [7] sorted and pruned the convolutional filters according to the ℓ_1 norm, which significantly reduced the computation cost while maintaining good accuracy. He et al. [8] proposed the filter pruning method (FPGM) based on the geometric median, which selects redundant filters through geometry rather than simple magnitude, so as to obtain a more stable pruning effect in different network architectures. Together, these studies point out an important problem: there may be a significant difference between the reduction of FLOPs and the inference speedup on real devices under bandwidth-limited hardware.

III. EXPERIMENTAL SETUP

A. Hardware Platform

All device-side experiments were completed on NVIDIA Jetson Nano 2GB running JetPack 4.6. The device is equipped with a 128-core Maxwell GPU with a peak throughput of 472 GFLOPS (FP32) and a shared LPDDR4 memory bandwidth of 12.8 GB/s. The CPU is a quad-core ARM Cortex-A57 with a main frequency of 1.43 GHz. All inference experiments are completed using PyTorch 1.10, CUDA 10.2 and cuDNN 8.2. Performance analysis uses PyTorch's built-in `torch.profiler` with CUDA activity tracing enabled.

B. Dataset

We evaluate on the Cityscapes validation set [9]. The dataset contains 500 images collected from vehicle-mounted cameras, covering 50 European cities. The dataset defines a total of 30 semantic categories; according to standard practice, we report mIoU on 19 training categories and ignore the remaining invalid labels. The original Cityscapes image resolution is 2048×1024 ; during baseline inference, all images are scaled to 256×512 through bilinear interpolation (the input resolution used in model training).

C. Model Selection

We evaluated three candidate semantic segmentation models to select optimization targets. Table I summarizes the latency, throughput, parameter count and peak memory usage on Jetson Nano with an input resolution of 256×512 .

TABLE I
CANDIDATE MODEL COMPARISON ON JETSON NANO (256×512)

Model	Params (M)	Latency (ms)	FPS	Mem (MB)
FCN-ResNet50	32.96	795.4	1.3	259
DeepLabV3-MNV3	11.02	125.2	8.0	75
Fast-SCNN	1.14	28.8	34.8	75

Because the inference latency of FCN-ResNet50 reaches 795 ms (1.3 FPS), it cannot meet the real-time application requirements and is directly excluded. Although Fast-SCNN has achieved the highest throughput (34.8 FPS), its out-of-the-box accuracy is significantly lower than that of other models because it is not fine-tuned for Cityscapes. DeepLabV3-MobileNetV3 (DeepLabV3-MNV3) achieves the best balance: with only 75 MB of peak memory, it reaches 49.5% mIoU after fine-tuning and achieves an inference speed of 8.0 FPS.

More importantly, the comparison between FCN-ResNet50 and DeepLabV3-MNV3 shows that the inference latency on edge devices is not entirely determined by FLOPs: although the number of parameters of FCN-ResNet50 is only about 3 times that of the latter, its running speed is 6.3 times slower. This shows that in addition to the original computing power, memory bandwidth is also an important factor affecting the performance of edge inference. Therefore, we finally chose DeepLabV3-MNV3 as the optimization target.

IV. BASELINE PERFORMANCE ANALYSIS AND BOTTLENECK POSITIONING

A. Profiling Methodology

We use PyTorch `torch.profiler` with CUDA activity tracing enabled to perform performance analysis of DeepLabV3-MNV3 on Jetson Nano. The analysis script divides the inference pipeline into three stages: preprocessing (image loading, scaling and normalization), inference (model forward pass) and post-processing (performing argmax operation on the output logits). All time statistics are averaged from 200 timed inference iterations after 50 warm-up runs. In order to avoid time overlap between stages, CUDA synchronization is performed before the boundary of each stage.

B. Phase Breakdown

Table II and Fig. 2 show the wall-clock breakdown of a single inference pass at 256×512 .

TABLE II
INFERENCE PHASE BREAKDOWN (DEEPLABV3-MNV3, 256×512)

Phase	Time (ms)	Share (%)
Preprocessing	214.5	63.0
Inference	124.1	36.5
Postprocessing	1.6	0.5
Total	340.3	100

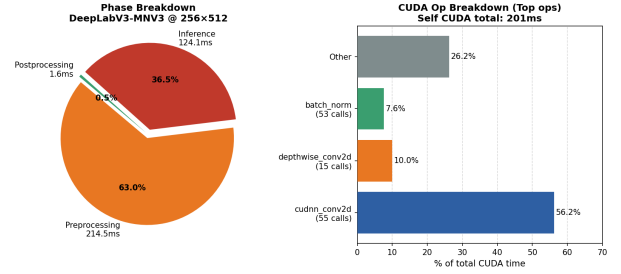


Fig. 2. Phase-level latency breakdown for DeepLabV3-MNV3 on the Jetson Nano. Preprocessing (image load, resize, and normalization) dominates at 63% of total wall-clock time.

Preprocessing dominates at 63% of total time, driven largely by the CPU-side image resize from 2048×1024 to 256×512 using Pillow. This result highlights that system-level preprocessing is a major bottleneck that is invisible to FLOPs-based analysis. Postprocessing (argmax) is negligible at 0.5%.

C. CUDA Operation Analysis

Within the inference phase, we rank operations by self CUDA time using `torch.profiler`. Table III lists the top operations.

TABLE III
TOP CUDA OPERATIONS BY GPU TIME (INFERENCE PHASE)

Operation	CUDA Time (ms)	Share (%)
cudnn_convolution	113.0	56.2
thnn_conv_depthwise2d_forward	20.1	9.9
cudnn_batch_norm	15.3	7.6
hardswish_	3.6	1.8
add_	3.1	1.5
Others	46.2	23.0
Total	201.2	100

Standard convolution (`cudnn_convolution`) accounts for 56.2% of GPU time, and depthwise convolution contributes another 9.9%, making convolutions collectively responsible for 66.1% of inference GPU time. Batch normalization adds 7.6%.

D. Bottleneck Identification

Two bottlenecks emerge from the profiling:

Preprocessing (system level). The CPU-side resize of high-resolution Cityscapes images accounts for 63% of wall-clock time. For real-world deployment with a camera feed at a fixed, lower resolution, this cost can be substantially reduced or eliminated.

Convolution under memory-bandwidth constraints (model level). Convolutions dominate GPU execution time, and the Jetson Nano’s Maxwell GPU has only 12.8 GB/s of shared memory bandwidth. Because memory transactions, not arithmetic throughput, limit convolution speed on this device, reducing FLOPs through pruning yields sublinear speedup: a 95.9% parameter reduction (pruning ratio 0.9) produces only a $2.63 \times$ latency improvement rather than the theoretically expected $\sim 10 \times$. This bandwidth-bottleneck insight guides both our resolution-scaling and pruning experiments.

V. RESOLUTION SCALING

A. Motivation

Resolution is an important means of weighing the inference speed and segmentation accuracy: halving the linear size of the input image will quadruply the number of pixels, thus directly reducing the amount of calculation required for preprocessing and convolution. However, since DeepLabV3-MNV3 is fine-tuned at a resolution of 256×512 , it can be evaluated at other resolutions and can be used to study the generalization ability of the model at different resolutions without retraining.

B. Method

We evaluate DeepLabV3-MNV3 at five input resolutions: 128×256 , 256×512 , 384×768 , 512×1024 and 768×1536 . At each resolution, the Cityscapes validation set image is scaled through bilinear interpolation before normalization; the label map uses nearest-neighbor interpolation to adjust to the same spatial size for mIoU calculation.

Latency is measured on Jetson Nano, and the average of 100 timed inference iterations is taken after 10 warm-up runs. mIoU is calculated on all 500 Cityscapes validation images.

C. Latency Results

Table IV reports latency and throughput at each resolution.

TABLE IV
RESOLUTION SCALING: LATENCY AND THROUGHPUT

Resolution	Latency (ms)	Std (ms)	FPS
128×256	54.9	2.1	18.2
256×512	124.9	0.4	8.0
384×768	262.1	0.8	3.8
512×1024	457.5	0.9	2.2
768×1536	990.5	1.4	1.0

Latency increases roughly with the increase in the number of pixels (area): when it is increased from 128×256 to 256×512 (the number of pixels increases by 4 times), the latency increases by $2.3 \times$; and from 256×512 to 768×1536 (the number of pixels increases by 9 times), the latency increases by $7.9 \times$, which once again reflects the impact of the memory bandwidth bottleneck.

D. Accuracy–Efficiency Tradeoff

Fig. 3 shows inference latency and speedup across the five resolutions.

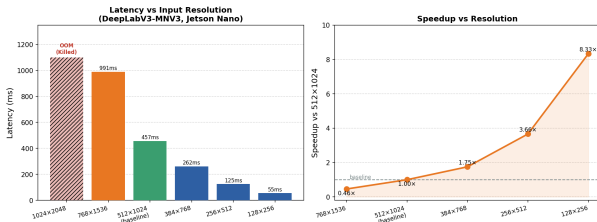


Fig. 3. Inference latency (ms) and speedup relative to the 256×512 baseline across five input resolutions on the Jetson Nano.

TABLE V
RESOLUTION SCALING: mIoU ON CITYSCAPES VALIDATION

Resolution	mIoU (%)	FPS
128×256	34.9	18.2
256×512	49.5	8.0
384×768	48.8	3.8
512×1024	43.7	2.2
768×1536	33.6	1.0

Table V lists mIoU at each resolution.

mIoU peaks at training resolution (256×512 , 49.5%) and declines at higher or lower resolution. When the resolution is higher than 256×512 , the accuracy of the model decreases, because the model has never been trained on such an input scale, and the fixed atrous rates cannot meet the needs of a larger receptive field. Reducing the resolution will lose the spatial details required for the fine category boundary. The mIoU with a resolution of 384×768 reaches 48.8%, which is only 0.7 percentage points lower than the best result, but the inference speed drops to 3.8 FPS, which is about $2.1 \times$ slower than the training resolution. For application scenarios with higher accuracy requirements than frame rate, 384×768 provides a more practical operating point. For real-time applications, the resolution of 128×256 can reach 18.2 FPS, but it requires a significant accuracy cost (34.9% mIoU, a decrease of about 14.6 percentage points). In general, the experimental results show that the training resolution of the model is also its best inference resolution, and the resolution scaling higher than the training size does not bring additional accuracy benefits to the architecture.

VI. FPGM STRUCTURAL PRUNING

A. Method

We apply Filter Pruning Via Geometric Median (FPGM), proposed by He et al. [8], to reduce the parameter count and inference latency of DeepLabV3-MNV3. FPGM identifies redundant filters by computing, for each layer, the geometric median of all filters in weights space. Filters closest to the geometric median are considered most redundant, as their feature representations can be approximated by neighboring filters, and will be removed. This criterion is more stable than magnitude-based pruning, which can incorrectly discard small-magnitude filters that encode unique features.

We implement FPGM using torch_pruning library [10], which constructs a dependency graph of the model and propagates channel changes through coupled layers automatically. This is essential for MobileNetV3's inverted residual blocks, where removing output channels from one layer requires corresponding changes to the next layer's input channels.

B. Pruning Configuration

Layer selection. We prune only pointwise Conv2d layers (kernel size 1×1). Depthwise convolutions are excluded because each filter maps directly to one input channel, therefore removing any filter would break the channel pairing with the

preceding pointwise layer. Layers with fewer than 16 output channels are also excluded to prevent causing destabilization of the early feature extraction layers that the entire network depends on. The final classifier head (out_channels = 19) is excluded to preserve all class outputs.

Pruning ratios. We sweep four pruning ratios: 30%, 50%, 70%, and 90%, all uniformly applied across all selected layers. Table VI reports the resulting parameter and FLOPs counts.

TABLE VI
PRUNED MODEL STATISTICS (BEFORE FINE-TUNING)

Ratio	Params (M)	Params↓	FLOPs (G)	FLOPs↓
0%	11.025	0.0%	324.93	0.0%
30%	6.023	45.4%	306.57	5.6%
50%	3.522	68.1%	297.66	8.4%
70%	1.657	85.0%	291.26	10.4%
90%	0.453	95.9%	287.50	11.5%

A key observation is that FLOPs reduction is substantially smaller than parameter reduction across all ratios — at most 11.5% versus 95.9%. This disparity arises from the architecture of MobileNetV3: depthwise convolutions, which are excluded from pruning, account for the majority of FLOPs. Pointwise convolutions, which are pruned, hold most parameters but contribute relatively few FLOPs due to their 1×1 kernel size. The consequences of this asymmetry on actual inference latency are discussed in Section VII.

C. Deployment Pipeline

Pruning is performed offline on an NVIDIA A100 GPU to avoid the memory and compute constraints of the Jetson Nano. The pipeline proceeds as follows:

- **Prune.** The pretrained DeepLabV3-MNV3 checkpoint is loaded and pruned at each target ratio using FPGM. A dummy input tensor is used solely for dependency graph tracing; the actual weight removal is resolution-agnostic.
- **Fine-tune.** Each pruned model is fine-tuned on 2,900 Cityscapes training images for 20 epochs using SGD with learning rate 0.005, momentum 0.9, and cosine annealing schedule. Fine-tuning is conducted at 256×512 to match the original training resolution.
- **Export.** Pruned and fine-tuned state dictionaries are exported and transferred to the Jetson Nano. The pruned architecture is reconstructed on-device from a stored channel specification without requiring torch_pruning at inference time.
- **Benchmark.** Inference latency is measured on the Jetson Nano at 256×512 using 10 warmup iterations followed by 50 timed runs with a synthetic input tensor, isolating model computation from preprocessing variance.

D. Accuracy Evaluation

mIoU is evaluated on all 500 Cityscapes images in validation set at 256×512 resolution. One-shot pruning without fine-tuning collapses accuracy to near zero at all pruning ratios, confirming that fine-tuning is a required step rather than

an optional refinement. Full accuracy results are reported in Section VII.

VII. RESULTS AND DISCUSSION

A. Latency and Speedup

Table VII reports inference latency and speedup on the Jetson Nano at 256×512 across all pruning ratios. Speedup is computed relative to the unpruned baseline.

TABLE VII
FPGM PRUNING: LATENCY AND SPEEDUP ON JETSON NANO (256×512)

Ratio	Params (M)	Params↓	FLOPs↓	Latency (ms)	Speedup
0%	11.025	0.0%	0.0%	199.95	1.00×
30%	6.023	45.4%	5.6%	144.82	1.38×
50%	3.522	68.1%	8.4%	114.85	1.74×
70%	1.657	85.0%	10.4%	89.85	2.23×
90%	0.453	95.9%	11.5%	76.15	2.63×

B. The FLOPs–Latency Gap

The most significant finding from Table VII is the large disparity between FLOPs reduction and actual latency reduction. At 90% pruning ratio, FLOPs are reduced by only 11.5%, yet latency drops by 61.9%, yielding a 2.63× speedup. Conversely, a naive FLOPs-proportional prediction would anticipate only an 11.5% latency reduction — roughly 1.13×. The observed speedup is more than twice this prediction.

This gap is explained by the memory-bandwidth bottleneck identified in Section IV. On the Jetson Nano’s Maxwell GPU, with only 12.8 GB/s of shared memory bandwidth, the dominant cost of convolution is not arithmetic throughput but the transfer of weight tensors from DRAM to compute cores. Fewer parameters directly reduces this transfer volume, relieving the bandwidth bottleneck and producing speedups that far exceed what FLOPs reduction alone would predict. This result demonstrates that on memory-bandwidth-constrained edge hardware, parameter count is a more reliable optimization target than FLOPs.

This finding also distinguishes our result from conventional pruning studies conducted on server GPUs. On high-bandwidth hardware such as the A100 (2,039 GB/s), weight transfer cost is amortized across large batches and overlapped with computation, making FLOPs the dominant cost. On Tegra X1, neither condition holds: batch size is one and the narrow memory bus is always the binding constraint.

C. Accuracy Results

Table VIII reports mIoU before and after fine-tuning for each pruning ratio.

Fine-tuning is essential. One-shot pruning without fine-tuning collapses mIoU to near zero at all pruning ratios — as low as 0.04% at 70% and 90% pruning. This confirms that structural pruning destroys the learned weight co-adaptations and that accuracy recovery requires fine-tuning regardless of pruning ratio. The contrast between pre- and post-fine-tuning

TABLE VIII
ACCURACY RESULTS: MIOU BEFORE AND AFTER FINE-TUNING
(256×512)

Ratio	Params↓	mIoU (no FT)	mIoU (FT)	Latency (ms)	Speedup
0%	0.0%	49.50%	49.50%	199.95	1.00×
30%	45.4%	2.17%	47.67%	144.82	1.38×
50%	68.1%	1.94%	44.45%	114.85	1.74×
70%	85.0%	0.04%	38.43%	89.85	2.23×
90%	95.9%	0.04%	27.64%	76.15	2.63×

results establishes fine-tuning as a required pipeline stage rather than an optional refinement.

Light pruning acts as regularization. At 30% pruning, fine-tuned mIoU reaches 47.67%, marginally below the unpruned baseline of 49.50% — a drop of only 1.83 percentage points despite removing 45.4% of parameters. This modest improvement over naive expectations is consistent with the known regularization effect of light structural pruning, which removes redundant filters and can reduce overfitting during fine-tuning on a limited dataset.

Three practical operating points emerge from Table VIII. The unpruned baseline maximizes accuracy at 49.50% mIoU with no compression overhead. At the other extreme, 90% pruning achieves the highest throughput at 13.1 FPS but incurs a 21.86 percentage point accuracy drop that renders safety-critical classes such as traffic lights and riders undetectable. The 50% pruned model represents the Pareto-optimal deployment point — it retains 44.45% mIoU with 68.1% fewer parameters and a 1.74× latency speedup, crossing the 8 FPS threshold suitable for many real-time edge applications. We recommend the 50% configuration as the default for bandwidth-limited edge deployment, with the unpruned baseline reserved for accuracy-critical scenarios and 90% pruning considered only where latency is the sole constraint and semantic detail is expendable.

D. Per-Class Sensitivity Analysis

Table IX reports per-class IoU for the unpruned baseline and all fine-tuned pruned models across all 19 Cityscapes classes.

TABLE IX
PER-CLASS IOU FOR SELECTED CLASSES ACROSS PRUNING RATIOS
(FINE-TUNED)

Class	0%	30%	50%	70%	90%
<i>Robust classes (large-area)</i>					
Road	95.04	94.85	94.42	93.97	92.28
Sky	85.00	84.37	83.46	81.92	77.29
Vegetation	82.92	82.53	81.83	80.43	76.54
<i>Sensitive classes (small/safety-critical)</i>					
Traffic light	21.70	18.79	14.50	3.31	0.00
Rider	24.78	20.38	15.46	4.72	0.00
Motorcycle	21.99	19.34	11.43	1.33	0.00

Two distinct sensitivity patterns emerge from Table IX. Large-area classes such as road, sky, and vegetation retain high IoU even at 90% pruning as they occupy large contiguous

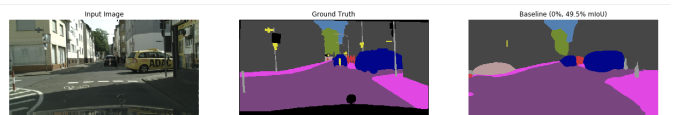


Fig. 4. Qualitative segmentation results for the unpruned baseline model (49.5% mIoU). Input image, ground truth, and model prediction shown left to right.

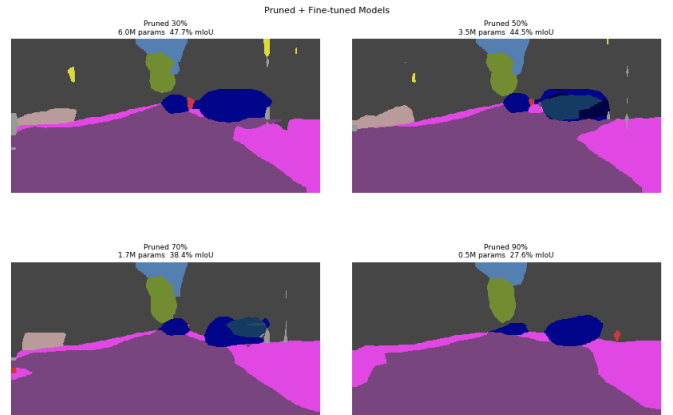


Fig. 5. Qualitative comparison of fine-tuned pruned models at four pruning ratios on the same input image. Large-area classes such as road and sky remain recognizable at 90% pruning, while fine-grained details degrade progressively.

regions with strong low-frequency texture signals that survive aggressive filter removal. In contrast, small and safety-critical classes including traffic light, rider, and motorcycle collapse sharply beyond 50% pruning, reaching 0.00% IoU at 90%, as the network loses capacity to detect subtle local features in small pixel regions. At pruning ratios of 70% and above, safety-critical classes degrade to near zero, making such configurations unsuitable for autonomous driving applications regardless of the latency benefit.

Figures 4 and 5 illustrate this pattern qualitatively on a representative Cityscapes image. At 30% and 50% pruning, large regions such as road, building, and sky remain visually coherent and close to the baseline prediction. At 70% and 90% pruning, fine-grained objects including traffic signs and poles progressively diminish and become entirely unrecognized, while background regions remain largely preserved. This visual evidence directly corroborates the per-class IoU collapse observed in Table IX.

E. Comparison with Original FPGM Results

He et al. [8] validate FPGM primarily on standard convolutional architectures, VGGNet and ResNet, on CIFAR-10 and ILSVRC-2012. On ResNet-110 with 40% pruning ratio, they report a 52.3% FLOPs reduction with negligible accuracy loss (0.16% drop after fine-tuning on CIFAR-10). On ResNet-50 with 30% pruning on ILSVRC-2012, FPGM achieves a 42.2% FLOPs reduction with only 0.56% top-1 accuracy drop. Notably, even in the original paper, a gap between theoretical

and realistic speedup is observed: ResNet-50 achieves 38.2% realistic latency reduction against 53.5% theoretical FLOPs reduction on a server GPU, suggesting that non-FLOPs costs already affect speedup even on high-bandwidth hardware.

Our application of FPGM to MobileNetV3 on Tegra X1 reveals a substantially more extreme version of this gap. At 90% pruning ratio, FLOPs are reduced by only 11.5%, which is far below the 42–52% achieved in the original work, yet latency drops by 61.9%, yielding a $2.63\times$ speedup. Two factors explain this divergence. First, the depthwise separable architecture of MobileNetV3 concentrates FLOPs in depthwise layers that cannot be pruned, limiting FLOPs reduction regardless of pruning ratio. Second, Tegra X1’s 12.8 GB/s shared memory bandwidth, which is approximately $160\times$ lower than server GPUs used in the original evaluation, means that parameter count, not FLOPs, determines inference speed. Together, these factors produce a regime where FPGM achieves larger practical speedup than its FLOPs reduction would suggest, the inverse of what standard efficiency metrics predict.

F. Challenges Faced

Our initial optimization plan included quantization-aware training (QAT), a well-established technique for INT8 inference acceleration. However, the Jetson Nano’s Maxwell GPU does not support INT8 inference through PyTorch, since integer arithmetic acceleration was only introduced in later Volta and Turing generation GPUs. Deploying a quantized model on Maxwell either falls back to FP32 or fails entirely, providing no latency benefit. This incompatibility forced us to redirect toward structural pruning, which produces standard dense models compatible with any inference backend.

VIII. FUTURE WORK

Two directions are identified as priorities for continued work. First, incorporating knowledge distillation during fine-tuning, using the unpruned model as a soft teacher, is expected to improve accuracy recovery at high pruning ratios, but requires additional training infrastructure and hyperparameter tuning beyond the scope of this project. Second, extending the training and fine-tuning pipeline to higher resolutions is necessary for deployment-realistic evaluation. All experiments in this work were conducted at 256×512 , which is significantly below the native Cityscapes resolution of 2048×1024 and below what practical autonomous driving systems require. A systematic sweep across training resolutions, with 256×512 , 512×1024 , and potentially 768×1536 , would characterize how the accuracy–efficiency tradeoff and the memory-bandwidth bottleneck evolve at deployment-realistic scales. Preliminary inference benchmarks suggest 512×1024 already introduces a $3.66\times$ latency increase relative to 256×512 , amplifying the bandwidth constraint further and making resolution a critical variable for any practical deployment study.

IX. CONCLUSION

This work investigates bottleneck-driven optimization of semantic segmentation on the NVIDIA Jetson Nano, addressing

the question of whether reducing FLOPs translates to real speedup on memory-bandwidth-constrained edge hardware. Rather than applying model compression blindly, we first profile the full inference pipeline to identify where time is actually spent, then apply optimizations informed by those findings.

Profiling reveals two distinct bottlenecks: CPU-side pre-processing accounts for 63% of wall-clock time under real deployment conditions, while GPU-side convolutions dominate inference execution under severe memory-bandwidth constraints. Resolution scaling confirms that the model’s training resolution (256×512) is also its optimal inference resolution — scaling above the training size provides no accuracy benefit while substantially increasing latency.

The central finding of this work concerns FPGM structural pruning. At 90% pruning ratio, parameter count is reduced by 95.9% and FLOPs by only 11.5%, yet latency drops by 61.9%, resulting in a $2.63\times$ speedup. This disparity demonstrates that on memory-bandwidth-bound hardware, parameter count is a more reliable optimization target than FLOPs: fewer parameters means less weight data transferred from DRAM to compute cores, directly relieving the binding memory constraint. This relationship does not hold on server GPUs where high bandwidth and large batch sizes amortize transfer costs, making our finding specific to the edge deployment regime.

The 50% pruned fine-tuned model is identified as the Pareto-optimal deployment point, retaining 44.45% mIoU with 68.1% fewer parameters and a $1.74\times$ speedup. Per-class analysis shows that large-area classes such as road and sky are robust to aggressive pruning, while safety-critical classes such as traffic lights and riders collapse at high pruning ratios, showing a practical constraint that limits aggressive compression for autonomous driving applications. Compared to resolution scaling, pruning offers a more favorable accuracy–efficiency tradeoff when the deployment resolution is fixed, while resolution scaling remains simpler when resolution flexibility is available.

More broadly, this work demonstrates that FLOPs and parameter counts are unreliable proxies for on-device latency on bandwidth-limited hardware. Profiling and empirical measurement on the target device are essential steps that cannot be replaced by theoretical efficiency estimates alone.

APPENDIX

AI TOOL USE DISCLOSURE

AI assistance (Claude, Anthropic) was used in two capacities during this project. For the final report, AI tools were used to polish prose and improve writing clarity on drafts written by the authors; all technical content, experimental results, and analysis were produced independently. For implementation, AI tools were used for clarification of library APIs and debugging assistance. All profiling interpretations, performance reasoning, and analytical conclusions are the authors’ own work.

TEAM CONTRIBUTIONS

Both authors contributed equally (50% each) to the overall research and writing.

Author	Share	Specific Contributions
Haotian Lei	~50%	Worked on report (Sections I–V), candidate model benchmarking, bottleneck profiling, mIoU evaluation, resolution scaling experiments, presentation slides.
Jiaming Liu	~50%	Worked on report (Sections VI–IX), FPGM pruning implementation, fine-tuning pipeline, pruning and finetuned accuracy and latency benchmarking, demo notebook, presentation slides.

REFERENCES

- [1] J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [2] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, “Rethinking atrous convolution for semantic image segmentation,” in *arXiv preprint arXiv:1706.05587*, 2017.
- [3] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan *et al.*, “Searching for mobilenetv3,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [4] R. P. Poudel, S. Liwicki, and R. Cipolla, “Fast-scnn: Fast semantic segmentation network,” in *arXiv preprint arXiv:1902.04502*, 2019.
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [6] M. Tan, R. Pang, and Q. V. Le, “Efficientdet: Scalable and efficient object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [7] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, “Pruning filters for efficient ConvNets,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [8] Y. He, P. Liu, Z. Wang, Z. Hu, and Y. Yang, “Filter pruning via geometric median for deep convolutional neural networks acceleration,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [9] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, “The cityscapes dataset for semantic urban scene understanding,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [10] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, “DepGraph: Towards any structural pruning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.